ЗВІТ ДО ПРАКТИЧНОЇ РОБОТИ № 36

«Серіалізація даних: робота з JSON»

Виконала: студентка групи І-23
Вовк Марія Віталіївна

1. Мета роботи

Навчитися перетворювати дані Python у формат JSON і назад, працювати з JSON-файлами, зберігати налаштування програми та обробляти помилки формату.

2. Вихідні дані

Для роботи використано власний файл витрат із практичної роботи 22. Він містить 13 записів за період 21.09.2026–27.09.2026. Загальна сума витрат — 1248,00 грн.

3. Хід виконання роботи

3.1. Завдання 1 — CSV → JSON

У task1.py файл expenses.csv читається за допомогою csv.DictReader. Суми перетворюються на числа float, а поле note записується в JSON як comment. Для збереження використано json.dump з indent=2, ensure_ascii=False та кодуванням UTF-8.

Результат запуску:

```
Maria Vovk, I-23\nrecords: 13\nsaved: data/expenses.json (розмір визначається програмою)
```

Створений файл data/expenses.json містить 13 об’єктів. JSON перевіряється командою python -m json.tool data/expenses.json.

3.2. Завдання 2 — прочитати, змінити, записати

У task2.py прочитано expenses.json, додано один запис із поточною датою, після чого видалено записи із сумою менше 10 грн. Початковий файл не змінюється. Результат зберігається в expenses_updated.json.

```
Maria Vovk, I-23\nadded: 1\ndeleted (amount < 10): 0\nrecords after update: 14\nsaved: data/expenses_updated.json
```

3.3. Завдання 3 — налаштування settings.json

У task3.py передбачено створення settings.json зі значеннями за замовчуванням, якщо файл відсутній. Якщо файл існує, програма читає налаштування та порівнює ліміт із сумою витрат. Помилка JSONDecodeError перехоплюється, номер рядка виводиться в sys.stderr, а програма завершується з кодом 1.

```
Перший запуск:\nMaria Vovk, I-23\nsettings.json not found, default created\n\nДругий запуск:\nMaria Vovk, I-23\nowner: Maria Vovk\ngroup: I-2
```

Пошкоджений файл не перезаписується, щоб не втратити налаштування.

3.4. Завдання 4 — серіалізація дати

Стандартний JSON не підтримує об’єкт datetime.date, тому початкова спроба викликає TypeError. Перший спосіб розв’язання — параметр default, який перетворює дату на ISO-рядок. Другий спосіб — власні функції encode_dates і decode_dates: перед записом дата перетворюється на рядок, а після читання відновлюється як date.

```
TypeError: Object of type date is not JSON serializable\ndefault result: {'owner': 'Maria Vovk', 'created': '2026-11-12'}\nround trip equal: True\nda
```

4. Висновок

Під час практичної роботи я навчилася перетворювати CSV-дані у JSON, читати та змінювати JSON-файли, зберігати налаштування програми й обробляти пошкоджений JSON. Також перевірила, що тип date потрібно перетворювати перед серіалізацією та відновлювати після читання.

5. Контрольні запитання

1. Що таке серіалізація і навіщо вона потрібна?

Серіалізація — це перетворення об’єктів програми у формат, який можна зберегти у файл або передати іншій програмі. Вона потрібна для обміну та збереження даних.

2. Які типи Python відповідають типам JSON? Що станеться з кортежем після запису й читання?

dict відповідає object, list — array, str — string, int і float — number, True/False — boolean, None — null. Кортеж після запису в JSON і читання перетвориться на список.

3. Чим json.dump відрізняється від json.dumps?

json.dump записує JSON безпосередньо у файловий об’єкт, а json.dumps повертає JSON як рядок.

4. Для чого параметри indent та ensure_ascii=False?

indent задає відступи для зручного читання JSON. ensure_ascii=False зберігає українські символи без перетворення на escape-послідовності.

5. Як обробити пошкоджений JSON-файл і чому його не можна мовчки перезаписати?

Потрібно перехопити json.JSONDecodeError і повідомити про помилку. Не можна мовчки перезаписувати файл, бо можна втратити збережені налаштування або дані.

6. Як серіалізувати тип, якого JSON не підтримує?

Можна використати параметр default у json.dump/json.dumps або власну функцію перетворення типу на підтримуваний JSON тип, а після читання відновити початковий тип.